

ContextCodec: Content-Focused Context Guidance for Ultra-Low Bitrate Speech Coding

Chengbin Liang¹, Wenqi Guo^{2,**}, Hao Cao¹, Zhijin Qin¹

¹ Department of Electronic Engineering, Tsinghua University, Beijing, China

² Department of Automation, Tsinghua University, Beijing, China

lcb25@mails.tsinghua.edu.cn

Abstract

Neural speech codecs enable low-bitrate speech communication, yet at ultra-low bitrates (< 1000 bps) preserving perceptual quality and intelligibility is challenging. Existing designs often prioritize acoustic details, leaving limited capacity for the core linguistic message under tight bitrate constraints. To address this, we propose ContextCodec, a codec that transmits content-focused context features to explicitly guide reconstruction. ContextCodec adopts a dual-branch encoder that decouples acoustic details from content-focused context. The context branch is trained with a CLIP-style contrastive loss that aligns context features with phoneme indices, reducing paralinguistic leakage. During decoding, these features are injected at each decoding stage for explicit guidance. In addition, we introduce a lightweight autoregressive latent refinement module. Experiments show a strong quality-intelligibility trade-off down to 500 bps, with an RTF of 0.4886 on a typical mobile CPU.

Index Terms: Neural speech codec, Ultra-low bitrate, CLIP-style phoneme alignment, Autoregressive latent refinement

1. Introduction

Neural speech codecs (NSC) aim to convert continuous speech signals into compact discrete representations while preserving perceptual quality and intelligibility. They are widely used in audio tokenization [1, 2, 3] and telecommunications applications [4, 5]. As speech communication is increasingly deployed in bandwidth-constrained environments such as satellite links, ultra-low-bitrate speech coding has become increasingly necessary [5, 6]. However, once the bitrate is pushed to the ultra-low regime, speech coding becomes a zero-sum bit-allocation problem: bits used to preserve how speech sounds inevitably reduce the capacity available to convey what is being said. As a result, maintaining both perceptual naturalness and intelligibility becomes fundamentally difficult.

Existing neural speech codecs can be broadly grouped into two families, as shown in Figure 1(a-b). Neural acoustic codecs [7] reconstruct speech waveforms through adversarial training [8, 9], with improvements in quantizer design [10, 11, 12], discriminator architectures [13], and structure [14]. While these methods achieve strong perceptual quality, their objectives are still dominated by acoustic fidelity, encouraging to preserve speaker timbre, or other fine acoustic details. Under an ultra-low-rate bottleneck, this can leave insufficient capacity for the linguistic content needed for accurate reconstruction, thereby hurting intelligibility.

Hybrid approaches incorporate semantic features from self-supervised learning (SSL) models [15, 16, 17] to strengthen se-

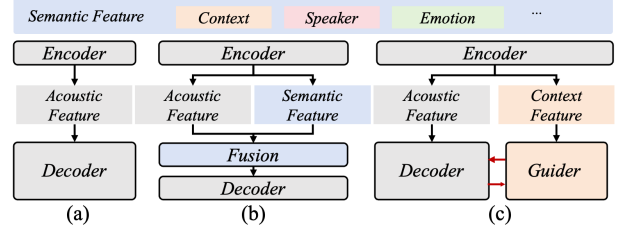


Figure 1: Different speech codec architectures. (a) Neural acoustic codecs. (b) Hybrid codecs. (c) ContextCodec(ours).

mantic integrity and downstream audio language model capabilities [18, 19, 20]. However, the semantic features are often not explicitly constrained to focus on linguistic content, and they may contain speaker identity and other paralinguistic attributes [21]. Moreover, in many hybrid designs, semantic cues are used primarily as initial priors; their influence can attenuate across successive decoding stages, leaving the final reconstruction still largely dominated by acoustic fidelity and vulnerable to intelligibility degradation at ultra-low bitrates [22, 23].

These limitations highlight the need for a content-first codec design at ultra-low bitrates, where the linguistic message must be prioritized over non-linguistic acoustic variation. Motivated by this, we propose ContextCodec, a context-guided neural speech codec for ultra-low bitrate speech communication (Figure. 1(c)). It adopts a dual-branch encoder that decouples acoustic-detail modeling from content-focused context modeling, and learns linguistically aligned context representations [24] to guide speech reconstruction. The learned context information is then used to support more reliable decoding under severe bitrate. We then inject the learned context features into each decoding stage to actively guide waveform reconstruction for improved intelligibility. Finally, inspired by learned image codecs [25, 26, 27], we introduce a lightweight autoregressive (AR) latent refinement module that progressively refines latents before decoding, enabling improved reconstruction quality. Our contributions are as follows:

1) We propose ContextCodec, a content-first neural speech codec designed for ultra-low-bitrate communication, which explicitly prioritizes linguistic content through decoupled acoustic and context modeling and a CLIP (Contrastive Language-Image Pretraining)-style linguistic alignment objective.

2) We design a context-guided attention decoder that preserves contextual guidance under tight bitrate constraints through acoustic pre-conditioning and stage-wise context injection, complemented by a lightweight autoregressive latent refinement module that improves reconstruction quality before waveform generation.

3) Extensive objective and subjective evaluations demonstrate that ContextCodec achieves a favorable qual-

** indicates the corresponding author.

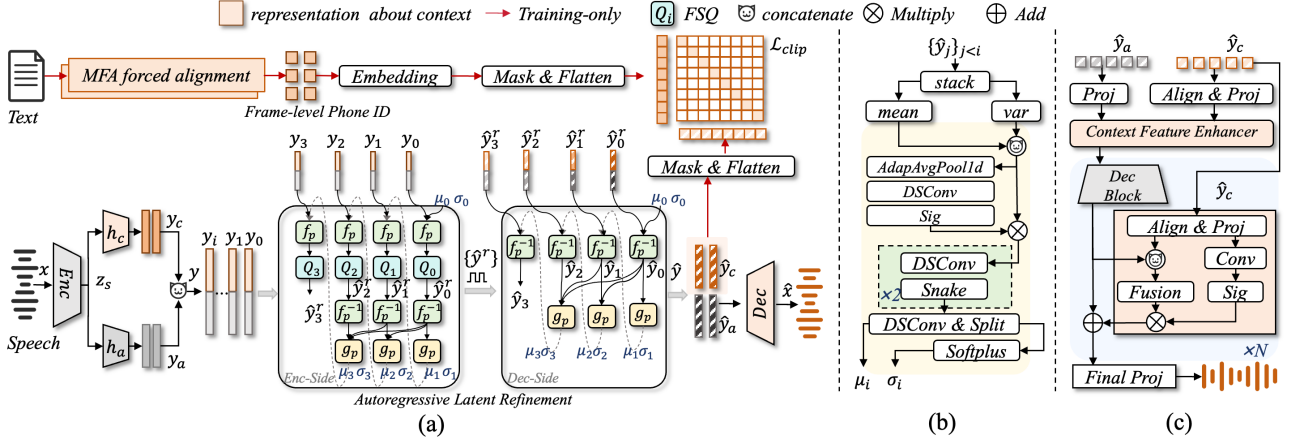


Figure 2: Overview of ContextCodec. (a) The end-to-end pipeline. (b) The predictor g_p . (c) The context-guided attention decoder.

ity–intelligibility trade-off down to 500 bps with a real-time factor (RTF) of 0.4886 on a typical mobile CPU.

2. Design of ContextCodec

ContextCodec is built upon a GAN-based quantized autoencoder framework with finite scalar quantization (FSQ) [28]. The base blocks of the shared encoder, decoder, and multiple discriminators follow the same architectural settings as DAC [10].

2.1. Overall architecture

As shown in Figure 2(a), ContextCodec maps an input waveform x to shared features $z_s = \text{Enc}(x)$ and applies a dual-branch encoder to obtain an acoustic stream $y_a \in \mathbb{R}^{B \times d_a \times T}$ and a context stream $y_c \in \mathbb{R}^{B \times d_c \times T}$; the concatenated latents are denoted by $y = [y_a; y_c]$, where B is batch size and T is the number of codec frames. The overall training pipeline includes three components. **(1) Autoregressive latent refinement:** y is partitioned into interleaved phases $\{y_i\}_{i=0}^{P-1}$, and a reusable predictor g_p estimates per-phase statistics for phase-specific quantization, yielding refined symbols $\hat{y}^r$ for transmission; on the decoder-side, the restored refined latents $\hat{y}$ are reconstructed from $\hat{y}^r$. **(2) CLIP-style phoneme alignment:** $\hat{y}$ is split into $\hat{y}_a$ and $\hat{y}_c$, and the quantized context codes are supervised by a frame-level contrastive loss against forced-aligned phoneme indices from text data. **(3) Context-guided attention decoding:** the decoder injects $\hat{y}_c$ into each decoding stage to reconstruct the waveform $\hat{x} = \text{Dec}(\hat{y}_a, \hat{y}_c)$.

2.2. Autoregressive latent refinement

To better allocate bits under tight budgets, we refine the concatenated latents $y = [y_a; y_c]$ before decoding by sequentially predicting per-phase mean/scale from already restored phases and quantizing normalized residuals, yielding refined symbols $\hat{y}^r$ (and restored $\hat{y}$) in a causal, lightweight manner. Specifically, we split y into P interleaved phase sequences $\{y_i\}_{i=0}^{P-1}$ (frames $t \equiv i \bmod P$) and process phases in order ($0 \rightarrow P-1$) within each P -frame block; we describe Enc-side bitstream generation and Dec-side reconstruction separately.

Enc-side. We denote the restored refined result of phase i by $\hat{y}_i$. For $i = 0$, we directly quantize with a phase-specific quantizer Q_0 : $\hat{y}_0^r = Q_0(y_0)$ and $\hat{y}_0 = \hat{y}_0^r$ (set $\mu_0 = 0, \sigma_0 = 1$). For each subsequent $i = 1, \dots, P-1$, we use a reusable predictor g_p to estimate per-time mean and scale from the already restored sub-sequences, $(\mu_i, \sigma_i) = g_p(\{\hat{y}_j\}_{j < i})$. As shown in Figure 2(b), the module g_p aggregates the available

context by computing per-time mean and standard deviation across $\{\hat{y}_j\}_{j < i}$, applies adaptive average pooling followed by a small 1D convolution and sigmoid as a lightweight global gate, and then uses a small depthwise-separable 1D conv stack with Snake activations to output (μ_i, σ_i) . We then remove the predicted mean/scale to form a normalized residual:

$$\varepsilon_i = f_p(y_i; \mu_i, \sigma_i) = (y_i - \mu_i) \oslash \sigma_i, \quad (1)$$

We quantize ε_i with a phase-specific quantizer and restore it as:

$$\hat{y}_i^r = Q_i(\varepsilon_i), \quad \hat{y}_i = f_p^{-1}(\hat{y}_i^r; \mu_i, \sigma_i) = \mu_i + \sigma_i \odot \hat{y}_i^r. \quad (2)$$

where $\oslash$ and $\odot$ denote element-wise division and multiplication. The refined quantized sequence $\hat{y}^r$ (obtained by interleaving $\{\hat{y}_i^r\}_{i=0}^{P-1}$ back to the full rate) is then coded into a bitstream and transmitted to the decoder.

Dec-side. The decoder parses the received bitstream to recover the discrete symbols $\hat{y}^r$. It then reconstructs the restored refined latents $\hat{y}$ by repeating the same phase order ($0 \rightarrow P-1$): for $i = 0$, set $\hat{y}_0 = \hat{y}_0^r$; for $i \geq 1$, recompute $(\mu_i, \sigma_i) = g_p(\{\hat{y}_j\}_{j < i})$ from already restored phases and apply $\hat{y}_i = f_p^{-1}(\hat{y}_i^r; \mu_i, \sigma_i)$. Finally, interleaving $\{\hat{y}_i\}_{i=0}^{P-1}$ yields the full-rate $\hat{y}$, which is fed into the waveform decoder.

In a future streaming, stateful implementation, phases can be reconstructed in order by caching $\{\hat{y}_j\}_{j < i}$, requiring no extra fixed P -frame delay; the added algorithmic latency can remain one codec frame.

2.3. CLIP-style phoneme alignment

Inspired by Secousticcodec [21], we introduce a frame-level CLIP-style alignment objective that directly matches the codec context features with the paired phoneme sequence at the codec frame rate. We obtain frame-level phoneme IDs by Montreal Forced Aligner (MFA) forced alignment and map them to learnable vectors by an embedding table as illustrated in Figure 2(a). This objective is designed to encourage content-focused, frame-consistent context representations and to reduce speaker-related and other paralinguistic leakage in the context branch. Let $\hat{y}_c \in \mathbb{R}^{B \times d_c \times T}$ denote the frame-wise quantized context representation split from $\hat{y}$, and let $q \in \mathbb{N}^{B \times T}$ be the aligned phoneme indices. The embedding table maps q to $e \in \mathbb{R}^{B \times T \times d_e}$. We define a binary mask $m \in \{0, 1\}^{B \times T}$ to select valid frames and perform Mask&Flatten by dropping invalid frames and flattening only valid frame pairs into N samples. The CLIP-style contrastive objective is then computed as:

$$\ell_{ij} = \frac{\text{sim}(\tilde{y}_{c,i}, \tilde{e}_j)}{\tau}, \quad i, j = 1, \dots, N, \quad (3)$$

$$\mathcal{L}_{\text{clip}} = \frac{1}{2} \left(\text{CE}(\ell, t) + \text{CE}(\ell^\top, t) \right), \quad t_i = i, \quad (4)$$

where $\tilde{y}_c, \tilde{e}$ are ℓ_2 -normalized, $\text{sim}(\cdot, \cdot)$ is cosine similarity, and τ is temperature. The positive pair of sample i is the aligned (utterance, frame) pair $(\tilde{y}_{c,i}, \tilde{e}_i)$, while negatives are all other valid frames in the mini-batch.

2.4. Context-guided attention decoder

As shown in Figure 2(c), the context-guided attention decoder $\text{Dec}(\cdot)$ injects the quantized context representation $\hat{y}_c$ into waveform reconstruction, so that the context branch can actively guide generation. Rather than conditioning the decoder only at its input, we use context features to pre-condition the acoustic latents and to modulate each subsequent decoding stage, which helps preserve contextual guidance when reconstructing from heavily quantized representations.

Given the restored refined latents $\hat{y} = f_p^{-1}(\hat{y}^r)$, we split them along the channel dimension into an acoustic stream $\hat{y}_a$ and a context stream $\hat{y}_c$. Before upsampling, we apply a lightweight *context feature enhancer* to modulate $\hat{y}_a$ using $\hat{y}_c$ through two complementary pathways: (i) a global pathway that produces a channel-wise gate and (ii) a local pathway that predicts a time-varying residual. The gated and residual-modulated features are concatenated and fused by a small fusion network to produce context-conditioned acoustic features. The two pathways and the fusion network are realized with lightweight pointwise and kernel-3 1D convolutions with Snake activations.

We then progressively upsample the acoustic stream with transposed-convolution decoder blocks and residual 1D convolution units with Snake activations. At each decoding stage, the context stream is aligned to the current temporal resolution by linear interpolation and projected to the acoustic channel size with a pointwise convolution, which corresponds to *Align&Proj* in Figure 2(c). The projected context features also produce a sigmoid gate, denoted as *Sig*, and the stage-wise *Fusion* concatenates acoustic and projected context features and applies the same kernel-3 convolution, Snake, and pointwise convolution. The fused output is gated and added back to the acoustic features via a residual connection. The final waveform is produced by Snake, a kernel-7 1D convolution, and a *tanh* output layer.

2.5. Training objectives

We jointly optimize reconstruction, adversarial, and cross-modal alignment objectives. The generator loss is:

$$\mathcal{L}_G = \lambda_m \mathcal{L}_m + \lambda_{\text{adv}} \mathcal{L}_{\text{adv}}^G + \lambda_{\text{fm}} \mathcal{L}_{\text{fm}} + \lambda_{\text{clip}} \mathcal{L}_{\text{clip}}, \quad (5)$$

where $\mathcal{L}_m$ is the multi-scale mel loss, $\mathcal{L}_{\text{adv}}^G$ and $\mathcal{L}_{\text{fm}}$ are the GAN generator and feature-matching losses. $\mathcal{L}_{\text{clip}}$ is a symmetric InfoNCE loss. The discriminator is optimized with $\mathcal{L}_D = \mathcal{L}_{\text{adv}}^D$.

3. Experiments

3.1. Experimental setup

Datasets and preprocessing. ContextCodec is trained on the training sets of LibriTTS [31] and AISHELL-3 [32]. To obtain frame-level supervision for alignment, we run an offline forced-alignment pipeline using MFA¹ to produce phoneme-level TextGrids [33]. We then convert phoneme time intervals

¹<https://montreal-forced-aligner.readthedocs.io/>

into a per-frame phoneme-id sequence at the codec frame rate. All training audio is segmented into 3-second clips. For evaluation, we test on VCTK [34] and ten languages from Common Voice 21.0 [35], including English, Chinese, German, French, Spanish, Russian, Arabic, Hindi, Japanese, and Korean to validate reconstruction performance and cross-lingual and cross-domain generalization. We randomly sample 6,000 VCTK utterances and 300 utterances per CV21 language. Each baseline runs inference at its native sampling rate, while all objective metrics are computed at 16 kHz for fair comparison.

Model and training. Training is performed at 16 kHz. The default model uses $N_e=4$ downsampling blocks with rates $[4, 4, 5, 8]$ and hop size $h = \prod r_i = 640$. Both heads output $d_a = d_c = 512$ channels; the context head uses $N_T=2$ Transformer layers with 4 attention heads. We choose $P=4$ phases and each phase has an independent FSQ quantizer with unequal vector. For 500 bps we use $\{[8 \times 6, 4 \times 2], [8 \times 6, 4 \times 1], [8 \times 6, 4 \times 1], [8 \times 6]\}$ and for 1000 bps we increase capacity to $\{[8 \times 12, 4 \times 4], [8 \times 12, 4 \times 2], [8 \times 12, 4 \times 2], [8 \times 12]\}$. Optimization uses AdamW with $\text{lr} 2 \times 10^{-4}$ and betas (0.8, 0.99), and we use GAN training with the same discriminators as DAC [10]. The temperature $\tau=0.07$ for CLIP-style alignment. We train for 1M steps on a single NVIDIA RTX 4090 GPU with batch size 8, using $\lambda_m=15$, $\lambda_{\text{adv}}=1$, $\lambda_{\text{fm}}=2$, $\lambda_{\text{clip}}=3$.

Baselines and metrics. We compare against representative baselines summarized in Table 1 using their official pretrained weights and recommended inference settings. For objective evaluation, we report signal-level metrics and the word error rate (WER) computed by a pre-trained Whisper-Turbo [36]. For each test language, we explicitly set Whisper-Turbo to the corresponding language when computing WER. For subjective evaluation, we conduct a preliminary pairwise preference listening test [37] with 15 listeners on 15 VCTK utterances sampled by stratified random selection, and all test outputs are loudness-normalized to -16 dB before playback.

3.2. Results

Objective results. Table 1 summarizes objective results. Overall, ContextCodec achieves strong performance on both English and multilingual test sets, offering a favorable trade-off between intelligibility and perceptual quality at ultra-low bitrates with a compact model size. These results also suggest reasonable generalization to unseen Common Voice languages, likely benefited by our phoneme-index supervision, which provides a shared pronunciation-based signal across different scripts. Nevertheless, phonological inventories differ across languages, and rare or unseen phonemes may be underrepresented in training.

Subjective results. We further conduct a subjective pairwise preference test as shown in Table 2, comparing against SemantiCodec, the strongest hybrid baseline in our low-bitrate objective comparison, and Opus as a widely used traditional codec: at 500 bps, ContextCodec is preferred over both baselines, indicating strong perceptual quality at ultra-low bitrates.

Deployment efficiency. Computational complexity² is evaluated in Table 3. For on-device evaluation, we report the end-to-end RTF on a Snapdragon 8 Gen 3 phone using CPU only and standard precision. Results show that ContextCodec remains efficient enough for practical deployment.

3.3. Ablation and analysis

As Table 4 shows, we investigate the contribution of each proposed component by comparing against several variants and de-

²GMACs is the maximum across the ptfllops, fvcure, and thop profiles. RTF is averaged over 10 runs on 10-second speech after warm-up.

Table 1: Objective evaluation results on a multilingual set and VCTK. Best results are shown in bold.

Model	Type	Bitrate	Param (M)	Sample rate	Multilingual (10 langs)				VCTK (en)			
					PESQ↑	STOI↑	SI-SDR↑	WER↓	PESQ↑	STOI↑	SI-SDR↑	WER↓
around 1000 bps Comparison												
EnCodec [13]	Acoustic	1500	14.85	24 kHz	1.629	0.814	0.511	45.76%	1.786	0.796	0.777	9.60%
DAC [10]	Acoustic	2000	74.18	16 kHz	1.201	0.727	-11.940	77.98%	1.245	0.740	-14.125	20.84%
SNAC [11]	Acoustic	994	19.80	24 kHz	1.835	0.834	-4.119	39.32%	2.384	0.861	0.115	5.28%
SecousticCodec [21]	Hybrid	1090	94.75	22 kHz	1.600	0.804	-26.675	56.14%	1.880	0.847	-25.572	9.40%
SemantiCodec [29]	Hybrid	1250	699.41	16 kHz	1.882	0.828	-25.758	30.95%	2.103	0.848	-23.693	4.62%
FACodec [30]	Hybrid	1600	102.33	16 kHz	1.310	0.743	-24.937	51.79%	1.623	0.788	-14.983	4.15%
SpeechTokenizer [23]	Hybrid	1000	103.68	16 kHz	1.242	0.715	-8.789	83.12%	1.351	0.747	-6.449	8.40%
X-Codec [18]	Hybrid	1000	169.33	16 kHz	1.846	0.812	-18.693	31.06%	2.257	0.822	-17.611	3.29%
Mimi [22]	Hybrid	1100	79.31	24 kHz	2.028	0.852	1.614	33.60%	2.256	0.840	2.968	4.57%
ContextCodec (ours)	Hybrid	1000	78.61	16 kHz	2.140	0.866	2.110	28.31%	2.476	0.880	3.614	2.25%
around 500 bps Comparison												
Mimi [22]	Hybrid	550	79.31	24 kHz	1.553	0.790	-5.517	60.35%	1.685	0.772	-4.285	10.22%
SpeechTokenizer [23]	Hybrid	500	103.68	16 kHz	1.150	0.590	-37.434	107.42%	1.210	0.645	-37.209	10.53%
SemantiCodec [29]	Hybrid	625	699.41	16 kHz	1.660	0.788	-26.761	52.69%	1.910	0.820	-24.128	11.42%
ContextCodec (ours)	Hybrid	500	78.61	16 kHz	1.758	0.812	-2.648	52.11%	2.120	0.846	-0.604	5.85%

Table 2: Subjective pairwise preference test results (%)

codec	preference (%)	codec	No pref. (%)
ContextCodec	97.92 $\leftarrow$ 2.08	Opus 6K [38]	0.00
ContextCodec	29.17 $\rightarrow$ 54.17	Reference	16.66
ContextCodec	52.92 $\leftarrow$ 40.83	SemantiCodec	6.25
Reference	66.67 $\leftarrow$ 20.83	SemantiCodec	12.50

Table 3: Complexity and Efficiency Comparison.

Model	GMACs $\downarrow$	RTF (A100) $\downarrow$	RTF (Android) $\downarrow$
ContextCodec	22.55	0.0029	0.4886
X-Codec	30.80	0.0027	-
Mimi	11.61	0.0049	-

note the variants from top to bottom as M_0 – M_7 : (i) M_1 replaces CLIP-style alignment with knowledge distillation from pre-trained Wav2Vec 2.0 [17]. We extract teacher representations by averaging the hidden states from *all* hidden layers, align them to codec frames, and match them to the student context features $\hat{y}_c$ with a cosine-similarity distillation loss weighted by $\lambda_{\text{distill}}=3$; (ii) M_2 and M_3 reduce the CLIP loss weight to 0.5 and 0; (iii) M_4 adopts the default decoder [10] with stage-wise context injection disabled; (iv) M_5 , M_6 , and M_7 vary the number of AR refinement phases P to 4, 2, and 0 (no refinement). It supports three conclusions. First, CLIP-style phoneme alignment provides more effective semantic supervision for intelligibility than distillation (M_0 vs. M_1). Second, explicit context guidance is beneficial, and the proposed stage-wise context injection mechanism is more effective than decoding from concatenated latents without guidance (M_0 vs. M_4). Third, AR refinement improves perceptual quality (M_5 vs. M_7).

Attribute predictability analysis. We quantify attribute predictability on TIMIT using linear probes for three supervision settings $M_0/M_1/M_3$ in Table 4. For each setting, we form two types of embeddings from the frame-level quantized context representation $\hat{y}_c$ for probing: an utterance-level embedding via time-averaging, and a phoneme-segment-level embedding using TIMIT boundaries (specifically for the Phone probe). We then train logistic-regression probes to predict phone, speaker, and dialect. Phone and Dialect probes are

Table 4: Ablation study on LibriTTS test-clean. “Enh.” denotes stage-wise context injection.

ID	Supervision	λ_{clip}	Enh.	P	PESQ↑	STOI↑	WER↓
M_0	Phoneme-CLIP	3.0	✓	4	2.048	0.892	5.56%
M_1	SSL-Distill[17]	–	✓	4	2.079	0.895	7.91%
M_2	Phoneme-CLIP	0.5	✓	4	2.114	0.900	9.14%
M_3	None	0	✓	4	2.100	0.899	10.58%
M_4	Phoneme-CLIP	3.0	×	4	2.080	0.896	8.20%
M_5	None	0.0	×	4	2.047	0.893	10.75%
M_6	None	0.0	×	2	1.963	0.884	10.51%
M_7	None	0.0	×	0	1.887	0.880	10.75%

Table 5: Attribute predictability analysis (accuracy, %).

ID	Supervision	Phone $\uparrow$	Speaker $\downarrow$	Dialect $\downarrow$
M_0	Phoneme-CLIP	88.7	51.8	26.6
M_1	SSL-Distill	70.2	91.0	28.2
M_3	None	66.5	82.2	30.8

trained on the TRAIN split and evaluated on the TEST split. For Speaker, since TRAIN and TEST contain disjoint speakers, we instead split each TRAIN speaker’s utterances into train and test sets and report accuracy on the test sets. As shown in Table 5, Phoneme-CLIP substantially improves Phone accuracy and reduces the predictability of Dialect and Speaker compared with SSL-Distill, indicating more content-specific representations with lower speaker/dialect leakage.

4. Conclusion

We propose ContextCodec, a context-guided neural speech codec for ultra-low bitrate speech communication. By prioritizing content preservation and using context as an explicit guidance signal during decoding, ContextCodec achieves a strong balance between intelligibility and perceptual quality at bitrates down to 500 bps. Extensive objective and subjective evaluations further demonstrate robust performance across English and multilingual test sets, together with an RTF of 0.4886 on a typical mobile CPU. Future work includes the streaming implementation and strengthening multilingual supervision for rare or unseen phonemes.

5. Acknowledgements

This work is supported by the National Key Research and Development Program of China under Grant No. 2023YFB2904300, the National Natural Science Foundation of China under Grant No. 62293484, and Beijing Natural Science Foundation (F251001).

6. Generative AI Use Disclosure

We used generative AI tools (GPT-5.2) to assist with English polishing and $\LaTeX$ formatting suggestions. All technical content, experimental results, and final wording were verified and approved by the authors.

7. References

- [1] Z. Borsos, R. Marinier, D. Vincent, E. Kharitonov, O. Pietquin, M. Sharifi, D. Roblek, O. Teboul, D. Grangier, M. Tagliasacchi *et al.*, “Audiolm: a language modeling approach to audio generation,” *IEEE/ACM transactions on audio, speech, and language processing*, vol. 31, pp. 2523–2533, 2023.
- [2] Y. Zheng, W. Tu, Y. Kang, J. Chen, Y. Zhang, L. Xiao, Y. Yang, and L. Ma, “Freecodec: A disentangled neural speech codec with fewer tokens,” in *Proc. Interspeech 2025*, 2025, pp. 4878–4882.
- [3] H. Hu, X. Zhu, T. He, D. Guo, B. Zhang, X. Wang, Z. Guo, Z. Jiang, H. Hao, Z. Guo, X. Zhang, P. Zhang, B. Yang, J. Xu, J. Zhou, and J. Lin, “Qwen3-tts technical report,” *arXiv preprint arXiv:2601.15621*, 2026.
- [4] W. B. Kleijn, A. Storus, M. Chinen, T. Denton, F. S. Lim, A. Luebs, J. Skoglund, and H. Yeh, “Generative speech coding with predictive variance regularization,” in *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2021, pp. 6478–6482.
- [5] J.-M. Valin and J. Skoglund, “Lpcnet: Improving neural speech synthesis through linear prediction,” in *ICASSP 2019-2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2019, pp. 5891–5895.
- [6] B. Zhang, I. McLoughlin, X. Miao, and A. Madhukumar, “Lsp-net: an ultra-low bitrate hybrid neural codec,” in *Proc. Interspeech 2025*, 2025, pp. 614–618.
- [7] N. Zeghidour, A. Luebs, A. Omran, J. Skoglund, and M. Tagliasacchi, “Soundstream: An end-to-end neural audio codec,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 30, pp. 495–507, 2021.
- [8] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial nets,” *Advances in neural information processing systems*, vol. 27, 2014.
- [9] Y. Zheng, W. Tu, L. Xiao, and X. Xu, “Srcodec: Split-residual vector quantization for neural speech codec,” in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 451–455.
- [10] R. Kumar, P. Seetharaman, A. Luebs, I. Kumar, and K. Kumar, “High-fidelity audio compression with improved rvqgan,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 27980–27993, 2023.
- [11] H. Siuzdak, F. Grötschla, and L. A. Lanzendörfer, “Snac: Multi-scale neural audio codec,” in *Audio Imagination: NeurIPS 2024 Workshop AI-Driven Speech, Music, and Sound Generation*.
- [12] D. Yang, S. Liu, R. Huang, J. Tian, C. Weng, and Y. Zou, “Hifi-codec: Group-residual vector quantization for high fidelity audio codec,” *arXiv preprint arXiv:2305.02765*, 2023.
- [13] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, “High fidelity neural audio compression,” *arXiv preprint arXiv:2210.13438*, 2022.
- [14] L. Xu, J. Wang, J. Zhang, and X. Xie, “Lightcodec: A high fidelity neural audio codec with low computation complexity,” in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 586–590.
- [15] S. Chen, C. Wang, Z. Chen, Y. Wu, S. Liu, Z. Chen, J. Li, N. Kanda, T. Yoshioka, X. Xiao *et al.*, “Wavlm: Large-scale self-supervised pre-training for full stack speech processing,” *IEEE Journal of Selected Topics in Signal Processing*, vol. 16, no. 6, pp. 1505–1518, 2022.
- [16] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, “Hubert: Self-supervised speech representation learning by masked prediction of hidden units,” *IEEE/ACM transactions on audio, speech, and language processing*, vol. 29, pp. 3451–3460, 2021.
- [17] A. Baevski, Y. Zhou, A. Mohamed, and M. Auli, “wav2vec 2.0: A framework for self-supervised learning of speech representations,” *Advances in neural information processing systems*, vol. 33, pp. 12449–12460, 2020.
- [18] Z. Ye, P. Sun, J. Lei, H. Lin, X. Tan, Z. Dai, Q. Kong, J. Chen, J. Pan, Q. Liu *et al.*, “Codec does matter: Exploring the semantic shortcoming of codec for audio language model,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 24, 2025, pp. 25697–25705.
- [19] H. Guo, F. Xie, K. Xie, D. Yang, D. Guo, X. Wu, and H. Meng, “Sococdec: A semantic-ordered multi-stream speech codec for efficient language model based text-to-speech synthesis,” in *2024 IEEE Spoken Language Technology Workshop (SLT)*, 2024, pp. 645–651.
- [20] X. Jiang, X. Peng, Y. Zhang, and Y. Lu, “Universal speech token learning via low-bitrate neural codec and pretrained representations,” *IEEE Journal of Selected Topics in Signal Processing*, vol. 18, no. 8, pp. 1477–1489, 2024.
- [21] C. Qiang, H. Wang, C. Gong, T. Wang, R. Fu, T. Wang, R. Chen, J. Yi, Z. Wen, C. Zhang, L. Wang, J. Dang, and J. Tao, “Secousticocdec: Cross-modal aligned streaming single-codecbook speech codec,” *arXiv preprint arXiv:2508.02849*, 2025.
- [22] A. Défossez, L. Mazaré, M. Orsini, A. Royer, P. Pérez, H. Jégou, E. Grave, and N. Zeghidour, “Moshi: a speech-text foundation model for real-time dialogue,” *arXiv preprint arXiv:2410.00037*, 2024.
- [23] X. Zhang, D. Zhang, S. Li, Y. Zhou, and X. Qiu, “Speechtokenizer: Unified speech tokenizer for speech language models,” in *The Twelfth International Conference on Learning Representations*.
- [24] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark *et al.*, “Learning transferable visual models from natural language supervision,” in *International conference on machine learning*. PMLR, 2021, pp. 8748–8763.
- [25] D. Minnen, J. Ballé, and G. D. Toderici, “Joint autoregressive and hierarchical priors for learned image compression,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.
- [26] Z. Cheng, H. Sun, M. Takeuchi, and J. Katto, “Learned image compression with discretized gaussian mixture likelihoods and attention modules,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 7939–7948.
- [27] D. He, Y. Zheng, B. Sun, Y. Wang, H. Qin, Y. Ren, X. Wang, H. Liu, M. Jin, and Y. Ma, “Channel-wise autoregressive entropy models for learned image compression,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 13016–13025.
- [28] F. Mentzer, D. Minnen, E. Agustsson, and M. Tschannen, “Finite scalar quantization: VQ-VAE made simple,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [29] H. Liu, X. Xu, Y. Yuan, M. Wu, W. Wang, and M. D. Plumbley, “Semanticocdec: An ultra low bitrate semantic audio codec for general sound,” *IEEE Journal of Selected Topics in Signal Processing*, 2024.

- [30] Z. Ju, Y. Wang, K. Shen, X. Tan, D. Xin, D. Yang, Y. Liu, Y. Leng, K. Song, S. Tang *et al.*, “Naturalspeech 3: zero-shot speech synthesis with factorized codec and diffusion models,” in *Proceedings of the 41st International Conference on Machine Learning*, 2024, pp. 22 605–22 623.
- [31] H. Zen, V. Dang, R. Clark, Y. Zhang, R. J. Weiss, Y. Jia, Z. Chen, and Y. Wu, “Libritts: A corpus derived from librispeech for text-to-speech,” *arXiv preprint arXiv:1904.02882*, 2019.
- [32] Y. Shi, H. Bu, X. Xu, S. Zhang, and M. Li, “Aishell-3: A multi-speaker mandarin tts corpus and the baselines,” *arXiv preprint arXiv:2010.11567*, 2020.
- [33] T. Mahrt, “Praatio,” <https://github.com/timmahrt/praatio>, 2016, gitHub repository.
- [34] J. Yamagishi, “English multi-speaker corpus for cstr voice cloning toolkit,” *URL <http://homepages.inf.ed.ac.uk/jyamagis/page3/page58/page58.html>*, 2012.
- [35] R. Ardila, M. Branson, K. Davis, M. Henretty, M. Kohler, J. Meyer, R. Morais, L. Saunders, F. M. Tyers, and G. Weber, “Common voice: A massively-multilingual speech corpus,” in *Proceedings of the 12th Conference on Language Resources and Evaluation (LREC 2020)*, 2020, pp. 4211–4215.
- [36] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in *International conference on machine learning*. PMLR, 2023, pp. 28 492–28 518.
- [37] M. Schoeffler, S. Bartoschek, F.-R. Stöter, M. Roess, S. Westphal, B. Edler, and J. Herre, “webmushra—a comprehensive framework for web-based listening tests,” *Journal of open research software*, vol. 6, no. 1, 2018.
- [38] J.-M. Valin, K. Vos, and T. Terriberry, “Definition of the opus audio codec,” Tech. Rep., 2012.